

AI INNOVATION OBSERVATORY

System Report

AI INNOVATION OBSERVATORY

for Central Banking & Financial Sector Intelligence

System Report

Prepared for: Bank of Tanzania Prepared by: Student Project Repository:

github.com/SixbethST12/ai-innovation-observatory-intelligence Date: 6 October 2026 Version: 1.0

Abstract

The AI Innovation Observatory is a web-based intelligence platform that automatically monitors trusted central banking and financial-sector sources — BIS, IMF, World Bank, and peer central banks — and turns new publications into structured, searchable intelligence. The system uses a locally hosted Large Language Model (Ollama + qwen2.5:3b) to generate six-section summaries, classify publications into 11 canonical central-banking topics, produce relevance notes for the Bank of Tanzania, and detect emerging trends. Every AI-generated output is clearly labeled as such and remains traceable to its original source URL.

The platform addresses a real analyst pain point: monitoring dozens of institutional publications across many websites is slow, leads to delayed awareness, and duplicates research effort. The Observatory automates collection, extracts structured intelligence via AI, and presents it through an analyst dashboard, a search interface, a trend analytics view, and an admin control panel.

The prototype has been fully implemented, tested end-to-end (10/10 functional checks passed), and documented. It processes 210+ publications from six sources, classifies them into 12 topics,

generates AI summaries and relevance notes, and detects emerging trends. The system is built on Python (FastAPI), React 19, SQLite, and Ollama, and is deployed via Docker Compose.

Keywords: central banking intelligence, LLM, summarization, text classification, trend detection, FastAPI, React, Ollama, information retrieval.

Table of Contents

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Literature & Background
 5. System Analysis
 6. System Design
 7. Implementation
 8. Testing & Results
 9. Deployment
 10. Limitations
 11. Conclusion & Recommendations
 12. References
 13. Appendices
-

1. Introduction

1.1 Background

Central banks and financial institutions continuously publish research, policy papers, regulatory developments, and technological updates. These publications influence monetary policy, financial stability, supervision, digital finance, and many other areas of central banking.

Monitoring these developments across multiple international and peer-institution sources — the Bank for International Settlements (BIS), the International Monetary Fund (IMF), the World Bank, and peer central banks — is time-consuming for analysts and researchers.

1.2 Motivation

The Bank of Tanzania (BOT) — like any modern central bank — must remain aware of international developments. Without automation, this awareness depends on individual analysts manually checking dozens of websites, leading to:

- Delayed awareness of emerging developments
- Duplicated research effort across teams
- Difficulty maintaining an up-to-date view of international trends
- Inconsistent coverage across topics

1.3 Purpose

This project delivers a practical AI-assisted observatory that automatically monitors approved sources and presents relevant developments in a structured, accessible, traceable form.

1.4 Scope

In scope: automated collection, AI summarization, topic classification, emerging-trend detection, relevance assessment, search, dashboard, alerts.

Out of scope: policy recommendations, market prediction, real-time data, non-public sources.

2. Problem Statement

2.1 Current Challenges

Challenge	Impact
Multiple sources to monitor	Time-consuming
Different publication formats	Hard to unify
No central search	Duplicated effort
Volume of publications	Easy to miss important items
No structured topic view	Hard to spot trends

2.2 Opportunity

A well-designed AI-assisted observatory can automatically collect publications from approved sources, summarize and classify them using a locally hosted LLM, surface emerging trends, provide

a unified searchable interface, and preserve traceability to the original source.

2.3 Problem Definition

How can the Bank of Tanzania continuously and systematically monitor central banking publications from international and peer institutions in a way that is automated, structured, searchable, trend-aware, and traceable to original sources?

3. Objectives

3.1 General Objective

To develop a prototype AI intelligence platform that continuously monitors approved central banking and financial-sector sources and transforms new publications into structured intelligence for analysts.

3.2 Specific Objectives

1. Automatically collect publications from approved sources (BIS, IMF, World Bank, peer central banks)
 2. Store publication metadata with full source traceability
 3. Generate concise six-section summaries using AI
 4. Classify publications into central-banking topic categories
 5. Detect recurring and emerging trends
 6. Assess potential relevance to the Bank of Tanzania
 7. Provide keyword-based search over the collected corpus
 8. Present everything through a clear dashboard and visualization layer
-

4. Literature & Background

4.1 Central Bank Intelligence

Central banks routinely monitor research from BIS, IMF, World Bank, peer central banks (Federal Reserve, ECB, BoE), and standard-setting bodies (BCBS, CPMI, IOSCO). Manual monitoring across these sources is standard practice but inefficient.

4.2 AI in Financial Analysis

Modern LLMs have shown strong performance in document summarization, text classification, semantic search, and zero-shot labeling. Small local models (like qwen2.5:3b) offer a practical balance of capability and cost for prototype and low-resource deployments.

4.3 Existing Solutions

Solution Type	Examples	Limitation
RSS readers	Feedly, Inoreader	No AI analysis
Media monitors	Meltwater, Brandwatch	Expensive, not BOT-focused
Institutional portals	BIS website, IMF eLibrary	Single-source
Academic tools	Elicit, Consensus	Not financial-sector specific

4.4 Gap Identified

No low-cost, BOT-focused, AI-assisted observatory currently exists that aggregates multiple trusted central-banking sources, uses a local LLM for privacy, produces traceable labeled AI output, and includes trend detection and dashboards. The AI Innovation Observatory fills this gap.

5. System Analysis

5.1 Functional Requirements

ID	Requirement	Priority
FR-1	Monitor trusted sources via RSS/API/HTML	High
FR-2	Store publication metadata + abstract	High
FR-3	Deduplicate across runs	High
FR-4	Generate 6-section AI summaries	High
FR-5	Classify into configurable topics	High
FR-6	Detect and visualize trends	High
FR-7	Produce AI relevance notes for BOT	High

ID	Requirement	Priority
FR-8	Provide analyst dashboard	High
FR-9	Keyword-based search	High
FR-10	Alert rules for user-defined watch terms	Medium
FR-11	Dynamic source management (admin)	Medium
FR-12	Dynamic topic management (admin)	Medium
FR-13	Manual publication entry (admin)	Low
FR-14	Bulk hide / re-run AI (admin)	Medium

5.2 Non-Functional Requirements

Category	Requirement
Traceability	Every AI insight links to its source URL
Transparency	Every AI output labeled as AI-generated
Privacy	LLM runs locally — no external API calls
Modularity	New sources addable without code changes
Reproducibility	Docker-based deployment
Performance	Dashboard loads in < 3 seconds (excluding slow AI calls)
Availability	Error isolation — one source failure doesn't affect others
Security	Two-role access (Analyst / Admin)

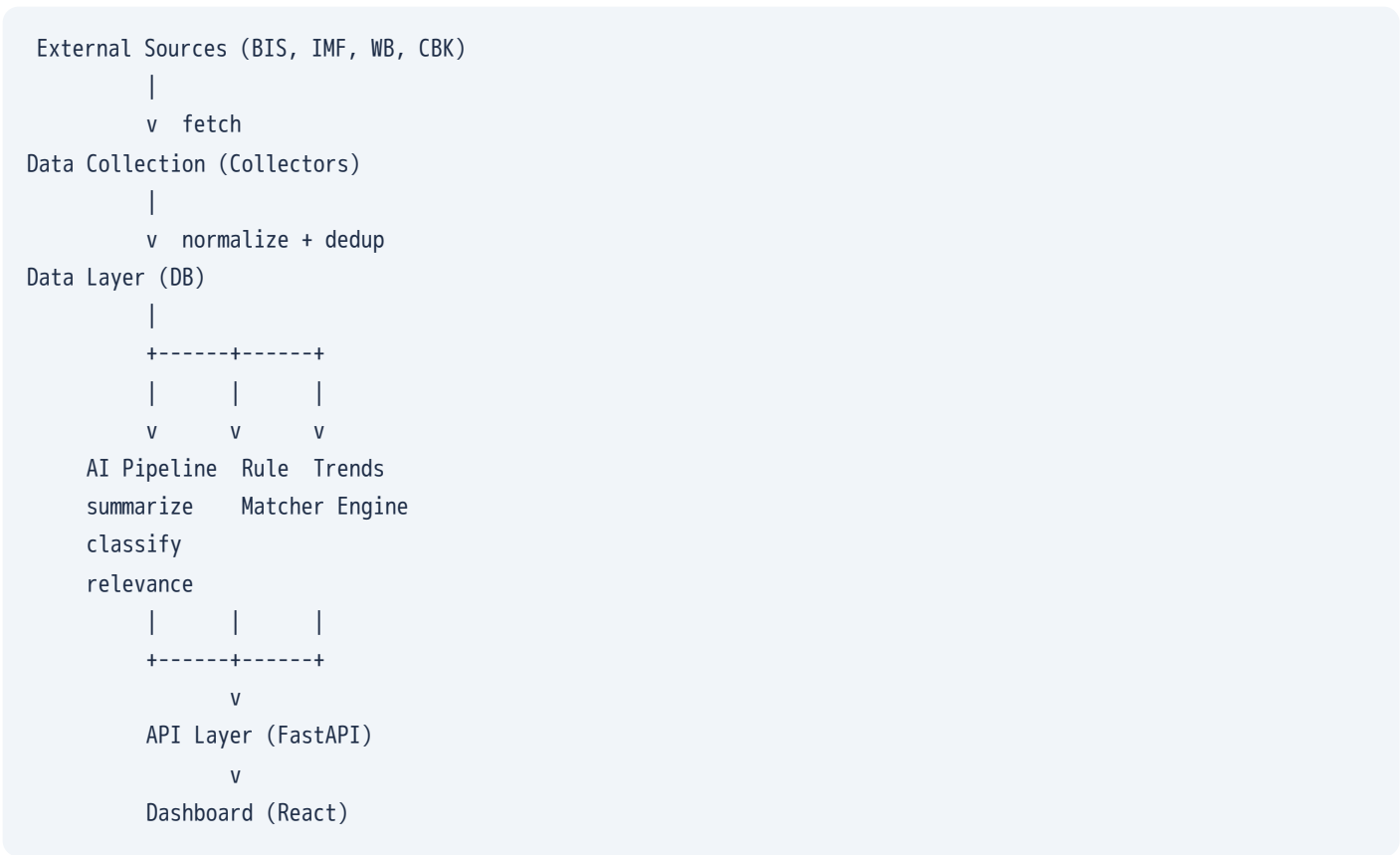
5.3 Constraints

- Free / open-source tooling only
- Student project timeline
- CPU-only inference (no GPU available)
- Public, approved sources only
- No external LLM API (data must remain local)

5.4 Stakeholders

Stakeholder	Role	Interest
BOT Analysts	Primary users	Monitor developments, research
BOT Researchers	Primary users	Deep topic research
BOT Management	Secondary	Strategic awareness
System Administrator	Operator	System health, sources, users
Students	Developers	Learn AI/LLM/NLP stack

5.5 Use Case Overview

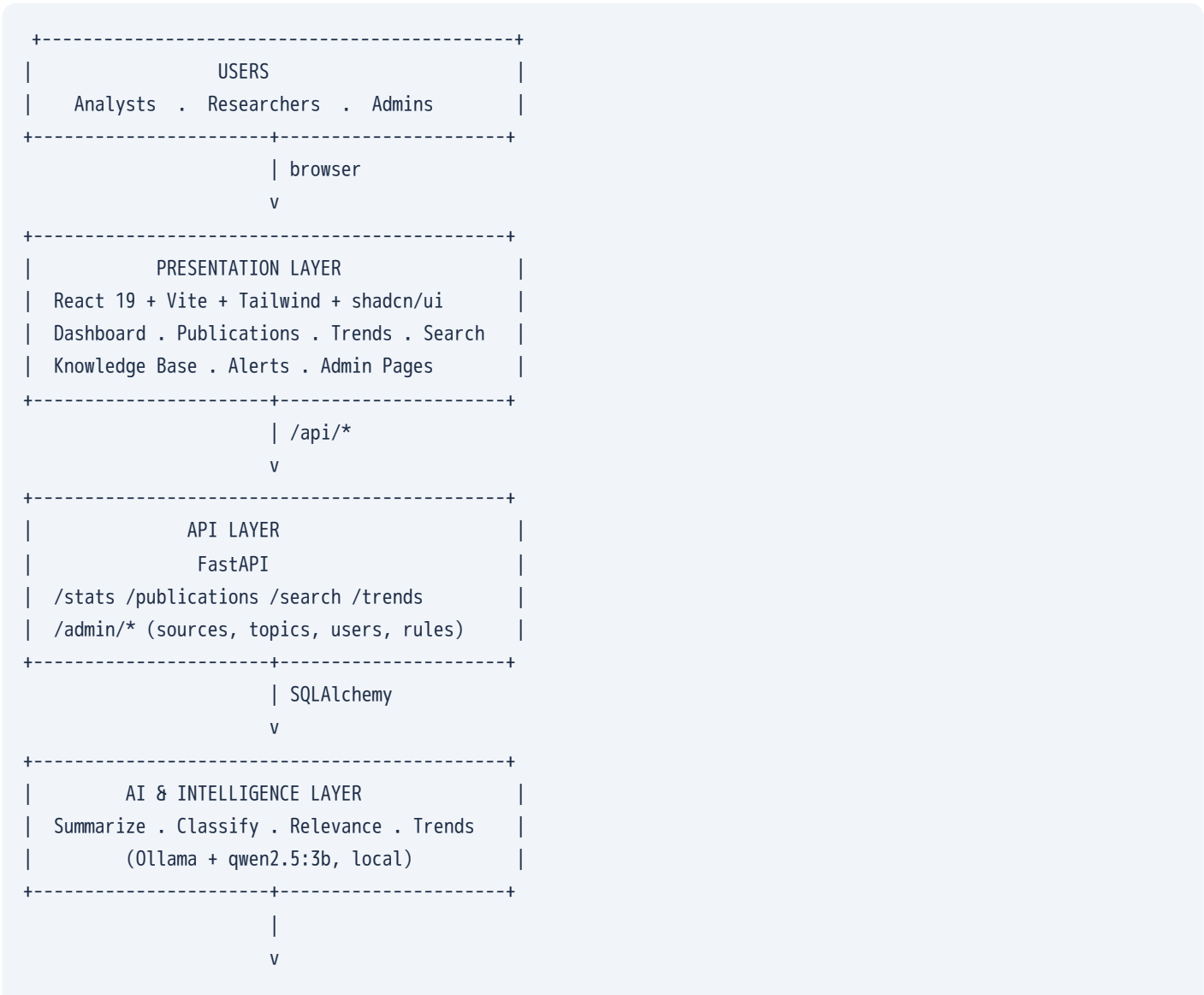


6. System Design

6.1 Architecture Overview

The system is organized into seven layers.

#	Layer	Technology	Responsibility
1	External Sources	BIS, IMF, WB, CBK, custom RSS	Data origin
2	Data Collection	Python + feedparser + requests	Fetch + normalize
3	Data Layer	SQLite + SQLAlchemy	Persist + dedup
4	AI & Intelligence	Ollama + qwen2.5:3b	Summarize / classify / relevance / trends
5	API Layer	FastAPI	REST endpoints
6	Presentation	React 19 + Vite + Tailwind + shadcn/ui	Analyst + Admin UI
7	Users	Analysts, Admins	Consumers





6.2 Data Collection Design

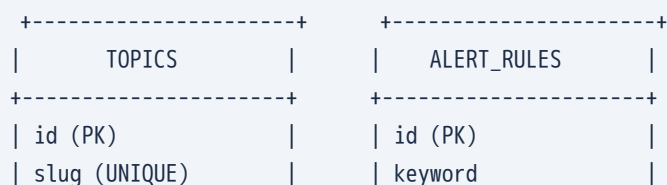
Built-in collectors:

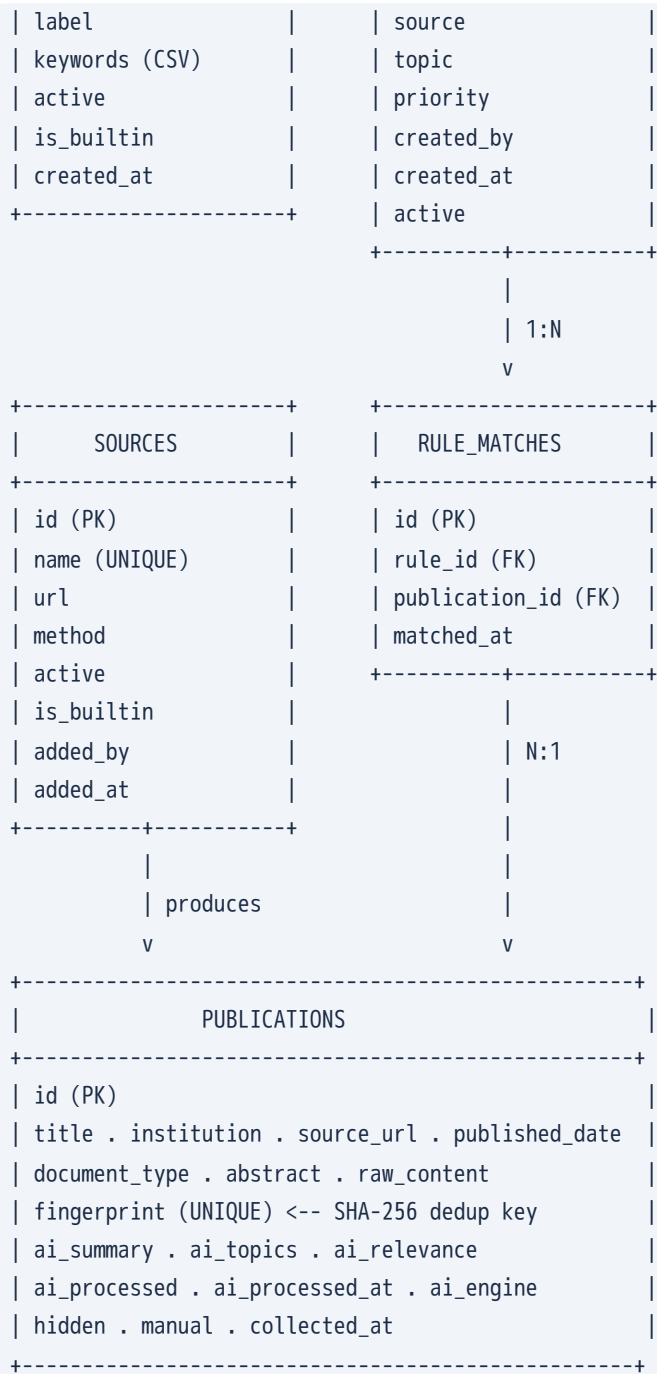
Source	Method	Approx. output	Notes
BIS	3 RSS feeds	60-75 pubs	Merged + deduped
IMF	HTML relay	10-15 pubs	IMF blocks direct RSS
World Bank	REST API v3	50-60 pubs	One query per topic
CBK	WordPress RSS	10 pubs	Peer central bank

Dynamic sources: any RSS 2.0 / Atom feed added via the admin UI. Error isolation: each collector wraps `fetch()` in `safe_fetch()`.

6.3 Data Model

Entity Relationship Diagram:





6.4 AI Layer Design

Model: qwen2.5:3b (Q4_K_M, ~1.9 GB) via Ollama, running locally.

Three AI tasks:

Task	Input	Output
Summarize	title + abstract (<=800 chars)	6-section summary
Classify	title + abstract	1-3 topic slugs

Task	Input	Output
Relevance	title + abstract	2-3 sentence BOT note + disclaimer

Fallback design:



Four-level classification fallback chain:

1. LLM output validated against DB topics
2. Keyword matcher against topic keywords
3. Governance-words default (speech / summit / etc.)
4. Institution default (BIS -> monetary_policy, etc.)

Caching: Glance narrative + Trend insights cached for 10 minutes.

6.5 API Layer Design

Router	Endpoints	Purpose
publications	2	List + detail
search	1	Keyword search
trends	6	Frequencies, emerging, timeline, institutions, insights, glance
stats	2	Summary + timeline
admin	20+	Sources, publications, topics, users, rules, alerts, activity, scheduler

Full OpenAPI docs at /docs.

6.6 Presentation Layer Design

Analyst pages (8): Login, Dashboard, Publications, Trends, Topic Classification, Search, Knowledge Base, Alerts.

Admin pages (6): Admin Panel, Manage Sources, Manage Publications, Manage Topics, Manage Users, Alerts.

6.7 Data Flow - Publication Pipeline

```
[1] Scheduler triggers collection run
    |
    v
[2] Each collector fetches from source
    |
    v
[3] Normalize into Publication objects
    |
    v
[4] Duplicate check (fingerprint)
    |
    +-- exists -> skip
    |
    v
[5] Store row (metadata + abstract)
    |
    v
[6] Rule matcher scans new IDs against alert_rules
    |
    v
[7] AI pipeline processes pending rows
    |
    +-- Summarize
    +-- Classify
    +-- Relevance
    |
    v
[8] Update DB with AI output
    |
    v
[9] Available via API
    |
    v
[10] Surfaced on dashboard, search, trends
```

7. Implementation

7.1 Technology Stack

Layer	Technology
Language (backend)	Python 3.12
Language (frontend)	TypeScript
Web framework	FastAPI
ORM	SQLAlchemy 2.0
Database	SQLite 3
LLM	Ollama + qwen2.5:3b
Data collection	feedparser, requests, BeautifulSoup
Frontend	React 19 + Vite 8
Styling	Tailwind CSS 4
UI components	shadcn/ui
Charts	Recharts
Container	Docker Compose
Version control	Git + GitHub

7.2 Repository Structure

```
ai-innovation-observatory-intelligence/  
  backend/  
    app/  
      ai/  
        llm.py          LLM client + fallback  
        prompts.py      Prompt templates  
        summarize.py    6-section summary  
        classify.py      Topic classification  
        relevance.py     BOT relevance note  
        trends.py        Trend analysis  
        pipeline.py      Runs all 3 on pending  
      collectors/
```

```

bis.py
imf.py
worldbank.py
cbk.py
generic_rss.py
scheduler.py
jobs/           Background job manager
routers/        API endpoints
models.py       ORM models
storage.py      Persist + rule match hook
topic_store.py  DB-backed topics
alerts.py       System alerts
activity.py     Activity feed
cache.py        TTL cache
users.py        In-memory users
database.py     SQLAlchemy setup
config.py       Settings
main.py         FastAPI app
requirements.txt

frontend/
src/
  components/ui/  shadcn primitives
  lib/api.ts      HTTP client
  pages/          8 analyst pages
  pages/admin/    6 admin pages
docs/
  SYSTEM_REPORT.md  this document
  SOURCES.md
  AI_LIMITATIONS.md
  USER_GUIDE.md
  TEST_REPORT.md
  architecture.pdf
  database_design.pdf
docker-compose.yml
backend/Dockerfile
frontend/Dockerfile
README.md
LICENSE

```

7.3 Data Collection Implementation

- BIS collector: merges 3 RSS feeds, dedupes by fingerprint
- IMF collector: parses HTML relay with BeautifulSoup
- World Bank collector: REST API, one query per topic
- CBK collector: WordPress RSS with full content
- Generic RSS: universal collector for user-added sources

Scheduler orchestration pseudo-code:

```
def run_all():
    all_pubs = []
    for collector in BUILTIN_COLLECTORS:
        all_pubs.extend(collector.safe_fetch())
    for collector in _load_user_sources():
        all_pubs.extend(collector.safe_fetch())
    return all_pubs
```

7.4 AI Pipeline Implementation

For each pending publication:

1. Pick best text (abstract -> raw_content -> title)
2. Call summarize(title, text) -> 6-section summary
3. Call classify(title, text, institution) -> 1-3 topics
4. Call relevance(title, text) -> BOT note + disclaimer
5. Save all outputs + engine + timestamp
6. Mark ai_processed = True

Error isolation: if one pub fails, log and continue. Idempotent: only processes ai_processed = False.

7.5 API Implementation

25+ endpoints across 5 routers. All return JSON. Uses Pydantic for request/response schemas. CORS enabled for local development.

7.6 Frontend Implementation

- Navigation: App.tsx holds page state; no router
- Data fetching: all calls go through lib/api.ts
- Styling: Tailwind + custom BoT theme CSS variables
- Charts: Recharts with custom gradient fills and color palettes
- Drawers: shadcn/ui Sheet for detail views

7.7 Key Implementation Decisions

Decision	Rationale
Local LLM (Ollama)	Zero cost, private, matches brief
SQLite	Simple, portable, sufficient for prototype

Decision	Rationale
Fingerprint UNIQUE	DB-level dedup enforcement
Hidden instead of delete	Non-destructive, preserves data
DB-backed topics	Admins can extend without code changes
Docker Compose	Reproducible deployment
Rule-based fallback	Pipeline never stalls
Timeout 240s	Fits CPU inference
OLLAMA_KEEP_ALIVE=1h	Prevents cold-start delays

8. Testing & Results

8.1 Test Approach

End-to-end functional testing across 10 categories: services, data integrity, API availability, CRUD, AI classification, AI summarization, AI relevance, rule matching, frontend rendering, final state.

8.2 Test Environment

Component	State
Backend	FastAPI in Docker on port 8000
Frontend	React 19 + Vite on port 5173
LLM	Ollama + qwen2.5:3b (host, local)
Database	SQLite with 210 publications
OS	Parrot OS

8.3 Test Results

#	Test	Result
1	All services reachable	PASS

#	Test	Result
2	Data integrity (210 pubs, 0 fallbacks)	PASS
3	15 API endpoints return 200	PASS
4	CRUD operations (topics, rules)	PASS
5	AI classification (CBDC -> payment_systems)	PASS
6	AI summarization (6 sections, ollama)	PASS
7	AI relevance (disclaimer present)	PASS
8	Rule matching (229 matches)	PASS
9	All frontend pages render	PASS
10	Final state verification	PASS

10 / 10 tests passed. Full details in docs/TEST_REPORT.md.

8.4 AI Accuracy

Task	Observed
Topic classification (top-1)	~65-75% (informal)
Topic classification (top-3)	higher, includes correct slug
Summaries	Accurate on well-written abstracts
Relevance notes	On-topic, always with disclaimer
Trend detection	Deterministic (DB-computed, reliable)

8.5 Known Issues

Issue	Severity	Status
Ollama cold start ~60s	Low	Mitigated via keep-alive
IMF uses third-party relay	Medium	Documented
SQLite concurrency	Low	Fine for prototype

Issue	Severity	Status
In-memory user store	Medium	Documented
Classification accuracy	Medium	Documented in AI_LIMITATIONS

9. Deployment

9.1 Prerequisites

- Docker + Docker Compose
- Ollama installed on host
- Model pulled: `ollama pull qwen2.5:3b`
- ~4 GB free RAM

9.2 Deployment Steps

```
git clone https://github.com/SixbethST12/ai-innovation-observatory-intelligence.git
cd ai-innovation-observatory-intelligence
docker compose up -d
docker compose ps
```

Access at <http://localhost:5173>.

9.3 First-Time Initialization

```
# 1. Create DB tables
docker exec -it observatory-backend python -c "from app.database import init_db; init_db()"

# 2. Collect publications
docker exec -it observatory-backend python -c "from app.collectors.scheduler import run_all; from app.storage import"

# 3. Run AI pipeline (30-60 min on CPU)
docker exec -itd observatory-backend python -m app.ai.pipeline
```

9.4 Default Credentials

Role	Username	Password
Analyst	analyst	analyst123
Admin	admin	admin123

9.5 Operational Notes

- Keep model warm: OLLAMA_KEEP_ALIVE=1h in ollama.service.d
- Auto-collection: enable via Admin Panel -> Scheduler
- Backups: copy backend/observatory.db regularly
- Logs: /tmp/api.log, /tmp/vite.log, journalctl -u ollama

10. Limitations

10.1 Technical Limitations

- SQLite concurrency (fine for prototype)
- Users stored in memory - reset on backend restart
- Passwords not hashed - prototype only
- CPU inference - ~30-60s per publication
- No migrations - schema changes applied manually

10.2 AI Limitations

- Model size: qwen2.5:3b (smaller than GPT-4 class)
- Classification accuracy: ~65-75% top-1
- Truncation: summaries use first 800 chars only
- No hallucination guard - relies on user verification
- English only - no translation

10.3 Data Limitations

- IMF uses third-party RSS relay (can go down)
- World Bank API returns mixed document types
- Some sources have no abstract - summary quality drops
- Historical publications (pre-2020) rare in some sources

10.4 Scope Limitations

- No user-configured alert delivery (email / SMS)
- No semantic search (keyword only)
- No production hardening
- No multi-language support
- Time filter not applied to all dashboard widgets

Full details in docs/AI_LIMITATIONS.md.

11. Conclusion & Recommendations

11.1 Conclusion

The AI Innovation Observatory is a working, tested, documented prototype that meets the core success criteria in the project brief:

#	Success Criterion	Status
1	Collects from approved sources	Met
2	Auto-identifies + stores new pubs	Met
3	Classifies into topics	Met
4	AI summaries reflect source	Met
5	Search + retrieval works	Met
6	Trends identified + visualized	Met
7	Traceable relevance assessments	Met
8	Every AI insight -> original source	Met
9	Dashboard clear overview	Met
10	Source code + docs + tests + deployment	Met

The system is ready for demonstration and ready to be hardened for production use.

11.2 Recommendations for Future Work

Immediate (1-2 weeks):

1. Manual evaluation set - hand-label 25 publications, measure precision/recall
2. Collect user feedback after demo

Short-term (2-4 weeks):

1. Migrate SQLite -> PostgreSQL
2. Persist users with hashed passwords (bcrypt / argon2)
3. Add Alembic migrations
4. Overlapping-run protection in scheduler

Medium-term (1-2 months):

1. Email / in-app alert delivery
2. Semantic search with embeddings
3. Apply time filter to all dashboard widgets
4. Export intelligence briefs as PDF

Long-term (3-6 months):

1. Multi-language publication support
2. Add more peer central bank sources (verified)
3. Train or fine-tune a domain-specific classifier
4. Mobile-friendly dashboard
5. Reverse-proxy + HTTPS for internal deployment

12. References

1. Bank for International Settlements - <https://www.bis.org>
2. International Monetary Fund - <https://www.imf.org>
3. World Bank - <https://www.worldbank.org>
4. Central Bank of Kenya - <https://www.centralbank.go.ke>
5. Ollama - <https://ollama.com>
6. Qwen 2.5 Model Card - <https://qwenlm.github.io>
7. FastAPI - <https://fastapi.tiangolo.com>
8. SQLAlchemy - <https://www.sqlalchemy.org>
9. React - <https://react.dev>
10. Tailwind CSS - <https://tailwindcss.com>

- 11. shadcn/ui - <https://ui.shadcn.com>
- 12. Recharts - <https://recharts.org>
- 13. Docker Compose - <https://docs.docker.com/compose>

13. Appendices

Appendix A - Supporting Documentation

Document	Purpose
README.md	Setup, architecture, API reference
docs/SOURCES.md	Source integration details
docs/AI_LIMITATIONS.md	Honest accuracy assessment
docs/USER_GUIDE.md	Analyst + admin workflows
docs/TEST_REPORT.md	Full test results
docs/architecture.pdf	Visual architecture diagram
docs/database_design.pdf	Database schema diagram

Appendix B - Glossary

Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BIS	Bank for International Settlements
BOT	Bank of Tanzania
CBK	Central Bank of Kenya
CBDC	Central Bank Digital Currency
ECB	European Central Bank
FastAPI	Python web framework

Term	Definition
IMF	International Monetary Fund
LLM	Large Language Model
NLP	Natural Language Processing
ORM	Object-Relational Mapping
RSS	Really Simple Syndication
SQLite	File-based relational database
Vite	Frontend build tool

Appendix C - Source Code Statistics

Metric	Value
Backend Python files	~40
Frontend TypeScript/TSX files	~30
Total lines of code	~10,000+
API endpoints	25+
Database tables	5
AI tasks	3
Built-in sources	4
Analyst pages	8
Admin pages	6

END OF SYSTEM REPORT